COMPUTER VISION

LAB SHEET :08

```
# Name: Deepti
# CUID:24250168
# Roll No.: 16   Sec:B

# Experiment No. 8
# Application of Optical Flow for Real-Time Object Tracking and Motion Analysis

# Import required libraries
import cv2
import numpy as np
import math

# Open the webcam
cap = cv2.VideoCapture(0)

# Check whether webcam is available
if not cap.isOpened():
    print("Error: Could not open webcam.")
    print("Check whether your camera is connected and available.")
    exit()

# Read the first frame
ret, old_frame = cap.read()

# Check whether the first frame was captured
```

```
if not ret:
    print("Error: Could not read the first webcam frame.")
    cap.release()
    exit()

# Convert the first frame to grayscale
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

# Shi-Tomasi Corner Detection parameters
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

# Detect feature points using Shi-Tomasi
p0 = cv2.goodFeaturesToTrack(
    old_gray,
    mask=None,
    **feature_params
)

# Check whether feature points were detected
if p0 is None:
    print("No feature points detected.")
    cap.release()
    exit()
```

```
# Lucas-Kanade Optical Flow parameters
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)

# Create a mask for drawing trajectories
mask = np.zeros_like(old_frame)

# Variables for motion analysis
total_distance = 0.0
total_motion = 0.0
frame_count = 0

# Start real-time tracking
while True:

    # Capture current webcam frame
    ret, frame = cap.read()

    # Stop if frame cannot be read
    if not ret:
```

```
    print("Error: Could not read webcam frame.")
    break

# Convert current frame to grayscale
frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

# Calculate Lucas-Kanade Optical Flow
p1, st, err = cv2.calcOpticalFlowPyrLK(
    old_gray,
    frame_gray,
    p0,
    None,
    **lk_params
)

# Check whether tracking was successful
if p1 is not None:

    # Select successfully tracked points
    good_new = p1[st == 1]
    good_old = p0[st == 1]

    # Store motion values for this frame
    frame_motion = []

    # Process every tracked point
    for new, old in zip(good_new, good_old):
```

```
# Get new and old coordinates
a, b = new.ravel()
c, d = old.ravel()
# Convert coordinates to integers
a, b = int(a), int(b)
c, d = int(c), int(d)

# Calculate horizontal displacement
dx = a - c
# Calculate vertical displacement
dy = b - d

# Calculate motion magnitude
magnitude = math.sqrt(
    dx * dx + dy * dy
)

# Calculate motion direction
angle = math.degrees(
    math.atan2(dy, dx)
)

# Store motion magnitude
frame_motion.append(magnitude)

# Add to total distance
total_distance += magnitude
```

```
    # Draw trajectory
    mask = cv2.line(
        mask,
        (a, b),
        (c, d),
        (255, 255, 255),
        2
    )

    # Draw tracked feature point
    frame = cv2.circle(
        frame,
        (a, b),
        4,
        (0, 255, 0),
        -1
    )

    # Draw displacement arrow
    frame = cv2.arrowedLine(
        frame,
        (c, d),
        (a, b),
        (0, 0, 255),
        2
    )

# Calculate average motion for current frame
```

```
if len(frame_motion) > 0:
    average_motion = np.mean(frame_motion)
else:
    average_motion = 0

# Add average motion to total
total_motion += average_motion

# Combine frame and trajectories
output = cv2.add(
    frame,
    mask
)

# Display title
cv2.putText(
    output,
    "Real-Time Object Tracking",
    (10, 30),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.8,
    (0, 255, 0),
    2
)

# Display tracking method
cv2.putText(
    output,
```

```
    "Lucas-Kanade Optical Flow",
    (10, 60),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (255, 255, 255),
    2
)

# Display number of tracked points
cv2.putText(
    output,
    f"Tracked Points: {len(good_new)}",
    (10, 90),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (255, 255, 255),
    2
)

# Display average motion
cv2.putText(
    output,
    f"Average Motion: {average_motion:.2f} pixels",
    (10, 120),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (255, 255, 255),
    2
```

```
    )

    # Display output
    cv2.imshow(
        "Lab Sheet 8 - Object Tracking",
        output
    )

    # Update feature points
    if len(good_new) > 0:
        p0 = good_new.reshape(-1, 1, 2)

    # Detect new points if too few points remain
    if len(good_new) < 10:

        p0 = cv2.goodFeaturesToTrack(
            frame_gray,
            mask=None,
            **feature_params
        )

        if p0 is not None:
            mask = np.zeros_like(frame)

# Calculate Farneback Dense Optical Flow
flow = cv2.calcOpticalFlowFarneback(
    old_gray,
    frame_gray,
```

```
    None,
    0.5,
    3,
    15,
    3,
    5,
    1.2,
    0
)

# Calculate motion magnitude and direction
magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

# Create HSV image
hsv = np.zeros_like(frame)

# Set saturation to maximum
hsv[..., 1] = 255

# Represent motion direction using hue
hsv[..., 0] = angle * 180 / np.pi / 2

# Represent motion magnitude using brightness
hsv[..., 2] = cv2.normalize(
    magnitude,
```

```
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

# Convert HSV to BGR
dense_flow = cv2.cvtColor(
    hsv,
    cv2.COLOR_HSV2BGR
)

# Add title to dense flow output
cv2.putText(
    dense_flow,
    "Farneback Dense Optical Flow",
    (10, 30),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.8,
    (255, 255, 255),
    2
)

# Display Farneback result
cv2.imshow(
    "Lab Sheet 8 - Farneback",
    dense_flow
)
```

```
# Update previous frame
old_gray = frame_gray.copy()

# Increase frame counter
frame_count += 1

# Wait for keyboard input
key = cv2.waitKey(30) & 0xFF

# Press R to reset tracking
if key == ord("r"):

    p0 = cv2.goodFeaturesToTrack(
        old_gray,
        mask=None,
        **feature_params
    )

    mask = np.zeros_like(old_frame)

    total_distance = 0.0
    total_motion = 0.0
    frame_count = 0

    print("Tracking points reset.")

# Press ESC to exit
```

```
    if key == 27:
        break

# Calculate final average motion
if frame_count > 0:
    final_average_motion = total_motion / frame_count
else:
    final_average_motion = 0

# Print final results
print()
print("Motion Analysis Results")
print("-----------------------")
print(f"Frames processed: {frame_count}")
print(f"Total tracked displacement: {total_distance:.2f} pixels")
print(f"Average motion: {final_average_motion:.2f} pixels/frame")

# Release webcam
cap.release()
# Close all OpenCV windows
cv2.destroyAllWindows()

# Observation
# Lucas-Kanade tracks selected feature points in real time.
# Shi-Tomasi detects strong corner points suitable for tracking.
# Green points represent tracked feature points.
# White lines represent the trajectory of moving points.
# Red arrows represent the direction of motion.
```

Farneback provides dense motion information for almost every pixel.

Fast movement, illumination changes, occlusion, and camera movement can affect accuracy.

Question 1: How does object tracking differ from object detection?

Object detection identifies objects and their locations in an individual frame.

Object tracking follows the movement of an identified object across multiple frames.

Question 2: Explain how Optical Flow can be used for real-time object tracking.

Optical Flow estimates the movement of pixels or feature points between consecutive frames.

The displacement of these points can be continuously calculated to track object movement

and estimate its trajectory in real time.

Question 3: What is the role of Shi-Tomasi Corner Detection in the Lucas-Kanade Optical Flow algorithm?

Shi-Tomasi detects strong corner points that are suitable for tracking.

These points are provided to Lucas-Kanade, which calculates their movement between frames.

Question 4: Why is Optical Flow suitable for motion analysis in videos?

Optical Flow estimates the movement between consecutive frames.

It provides information about direction, displacement, magnitude, and movement patterns.

Question 5: What challenges arise while tracking fast-moving or partially occluded objects?

Fast-moving objects may cause large displacement and motion blur.

Occlusion can hide feature points and cause tracking errors.

Lighting changes and background motion can also affect tracking accuracy.

Question 6: Compare Optical Flow-based tracking with deep learning-based object tracking methods.

Optical Flow tracking is lightweight and does not require a large training dataset.

Deep learning tracking methods can provide better object recognition and robustness,

but they usually require more computational resources and training data.

Question 7: How can Optical Flow be used in traffic monitoring and autonomous driving systems?

Optical Flow can estimate the movement of vehicles and pedestrians.

It can help determine motion direction, relative movement, traffic flow, and object movement.

Question 8: Explain the effect of camera motion on Optical Flow estimation.

Camera movement makes stationary objects appear to move in the video.

This produces additional optical flow and can make it difficult to distinguish

actual object motion from camera-induced motion.

Question 9: Mention five real-world applications where motion analysis using Optical Flow

is commonly employed.

Five applications are traffic monitoring, autonomous driving, video surveillance,

human activity recognition, and sports analytics.

Question 10: How can Optical Flow improve the performance of surveillance, robotics,

and human activity recognition systems?

Optical Flow provides continuous motion information.

It can help surveillance systems detect movement, robots understand environmental motion,

and human activity recognition systems analyze movement patterns.